

# Gradnja GUI

`java.awt`

`javax.swing`

1

## AWT (Abstract Windowing Toolkit)

Osnovni nabor grafičnih gradnikov,  
avtomatsko razporejanje gradnikov,  
obravnavanje dogodkov

Paket *awt* zagotavlja vmesnik do okenskega sistema –  
neodvisno od platforme.

2

## Tri skupine razredov v awt

- **Grafični gradniki (GUI Controls)**

Button, Canvas, Checkbox, CheckboxGroup, MenuItem, Choice, Component, Container, Dialog, FileDialog, Frame, Label, List, Menu, MenuBar, MenuComponent, MenuItem, Panel, ScrollBar, TextArea, TextComponent, TextField, Window, ScrollPane...

- **Upravljalci izgleda**

BorderLayout, CardLayout, FlowLayout, GridBagLayout, GridLayout

- **Graphics**

Color, Dimension, Event, Font, FontMetrics, Graphics, GridBagConstraints, Image, Insets, MediaTracker, Point, Polygon, Rectangle, Toolkit

3

## Grafični gradniki

- gradniki se nahajajo v **vsebnikih**

- vsebniki zagotavljajo metodo

**add (\_\_\_\_)**

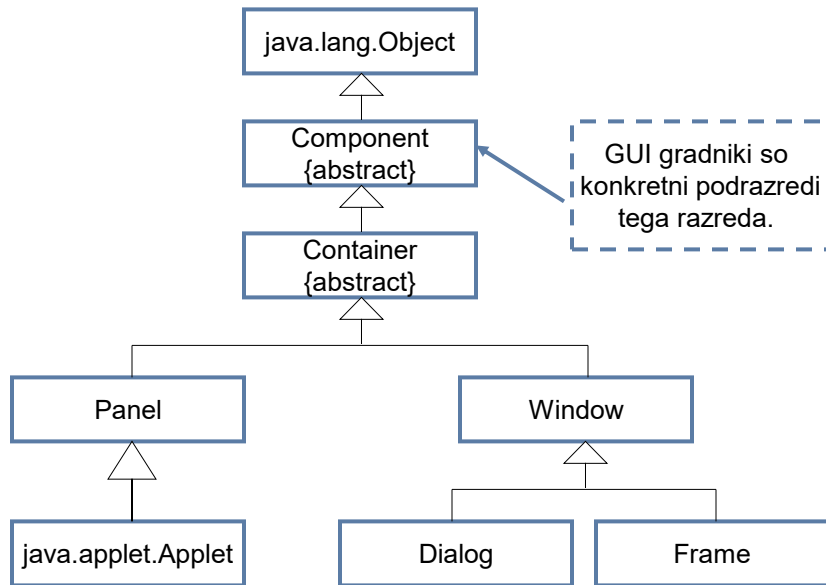
- položaj gradnikov določajo upravljalci izgleda (LayoutManager)

**setLayout (\_\_\_\_)**

- vsebnik v vsebniku

4

## Hierarhija



5

## Upravljalci izgleda

**FlowLayout, BorderLayout, CardLayout,  
GridLayout, GridBagLayout, BoxLayout**

6

## FlowLayout

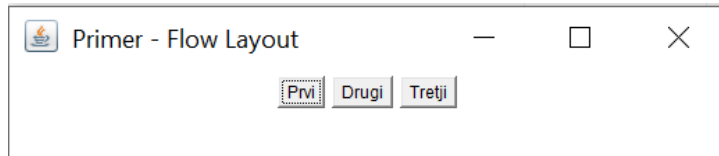
```
public class GUIFlowLayout {

    public static void main(String[] args) {

        Frame f = new Frame(" Primer - Flow Layout");
        f.setLayout(new FlowLayout());
        f.add(new Button ("Prvi"));
        f.add(new Button ("Drugi"));
        f.add(new Button ("Tretji"));

        f.pack();
        f.setVisible(true);

    }
}
```



7

## BorderLayout

```
import java.awt.*;

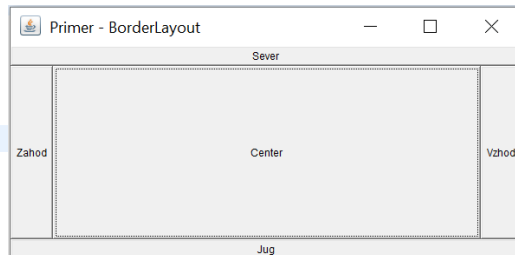
public class GUIBorderLayout {
    public static void main(String[] args) {

        Frame f = new Frame(" Primer - BorderLayout");
        f.setLayout(new BorderLayout());

        f.add(new Button ("Center"));
        f.add("North", new Button ("Sever"));
        f.add("South", new Button ("Jug"));
        f.add("East", new Button ("Vzhod"));
        f.add("West", new Button ("Zahod"));

        f.pack();
        f.setVisible(true);

    }
}
```



8

```
import java.awt.*; GridLayout

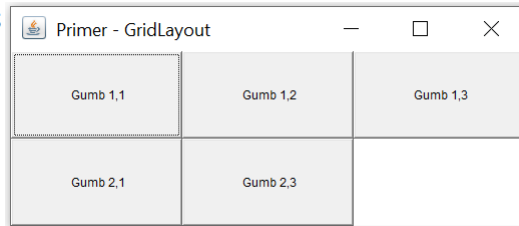
public class GUIGridLayout {

    public static void main(String[] args) {

        Frame f = new Frame(" Primer - GridLayout");
        f.setLayout(new GridLayout(2,3));

        f.add(new Button (" Gumb 1,1"));
        f.add(new Button (" Gumb 1,2"));
        f.add(new Button (" Gumb 1,3"));
        f.add(new Button (" Gumb 2,1"));
        f.add(new Button (" Gumb 2,3"));

        f.pack();
        f.setVisible(true);
    }
}
```



9

## CardLayout

```
public class GUICardLayout {

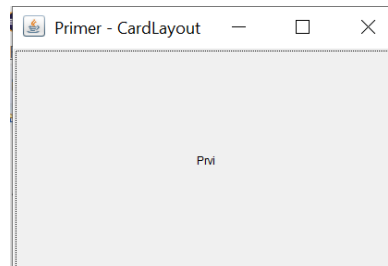
    public static void main(String[] args) | {

        Frame f = new Frame(" Primer - CardLayout");

        f.setSize(300,300);

        CardLayout cl = new CardLayout();
        f.setLayout(cl);

        f.add(new Button (" Prvi "));
        f.add(new Button (" Drugi"));
        f.add(new Button (" Tretji"));
        f.add(new Button (" Četrty"));
        f.add(new Button (" Peti"));
    }
}
```



10

## CardLayout-i

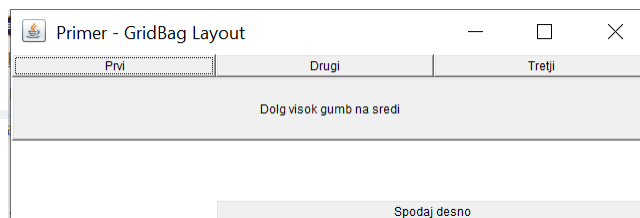
- CardLayout  
first(), next(), previous(), last(), show(kljuc)

11

## GridBagLayout

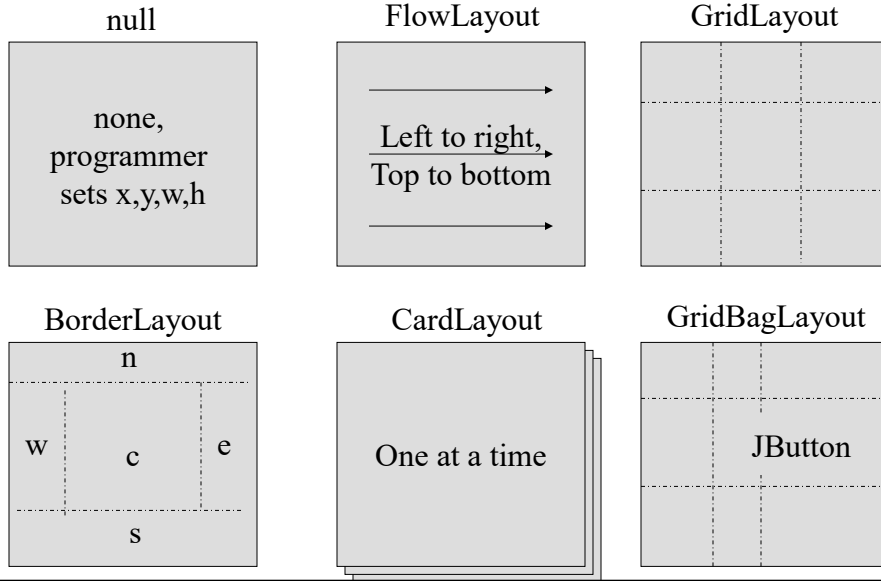
```
panel.setLayout(new GridBagLayout());
GridBagConstraints c = new GridBagConstraints();
c.fill = GridBagConstraints.HORIZONTAL;
gumb = new Button(" Prvi");
c.weightx = 0.5;
c.fill = GridBagConstraints.HORIZONTAL;
c.gridx = 0;
c.gridy = 0;
panel.add(gumb, c);

gumb = new Button(" Drugi ");
c.fill = GridBagConstraints.HORIZONTAL;
c.weightx = 0.5;
c.gridx = 1;
c.gridy = 0;
panel.add(gumb, c);
```



12

## Layout Manager Heuristics



13

## Dogodkovni model

14

## Obravnava dogodkov

- JDK 1.1. je vpeljal model, ki temelji na **delegiranju**
- obstaja več tipov dogodkov, definirani so v *java.awt.event* ter *javax.swing.event*
- komponente **se lahko** odzovejo na določene dogodke
- dogodki določenega tipa so posredovani nekemu objektu (poslušalcu), če je ta objekt registriran (prijavljen) za ta tip dogodkov na tem izvoru

15

```

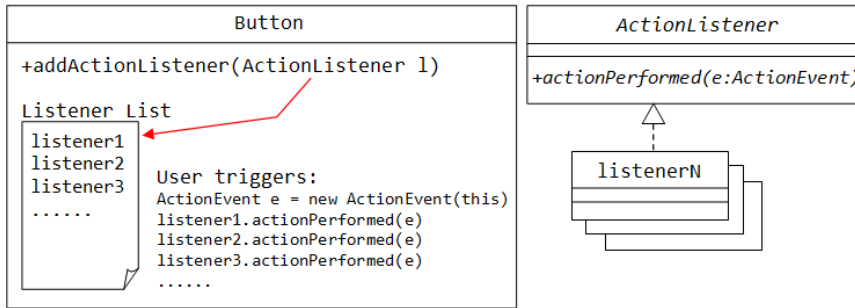
public class NoviPanel extends JPanel {
    public NoviPanel () {
        ...
        JButton b;
        add (b = new JButton("Gumb"));
        b.addActionListener (new ButtonListener());
    }
}

class ButtonListener implements ActionListener {
    public void actionPerformed (ActionEvent e)
    {
        System.out.println(...);
    }
}

```

16

## ActionListener



17

## Dogodkovni model v javi

- temelji na delegiranju (odzovejo se objekti "poslušalci")
- na viru dogodka je potrebno prijaviti/registrirati/dodati objekte poslušalce
- ti morajo ustrezati – zagotavljati ustrezne metode (predpisane z vmesniki)
- ko pride do dogodka, se vsem na seznamu prijavljenih pošlje obvestilo oz. proži ustrezna metoda

18

## Vsak dogodek - kreira se Event

- Vsak dogodek rezultira v kreiranju objekta *Event*, ki vključuje naslednje informacije
  - tip dogodka (npr. premik miške, pritisk tipke),
  - objekt, ki je bil "tarča" dogodka (npr. *JButton* ali *TextField*),
  - časovna oznaka pojavitve dogodka
  - lokacija, kjer se je pojavil, relativno na komponento, kateri je bil posredovan,
  - tipka, ki je bila pritisnjena (v primeru dogodka tipkovnice),
  - morebitne argumente,
  - stanje modifikatorskih tipk, ko sta *Shift* in *Control*.

19

### **dogodki**

- `ActionEvent`
- `WindowEvent`
- `MouseEvent`
- `KeyEvent`
- `ChangeEvent`
- `XXXEvent`

### **vmesniki**

- `ActionListener`
- `WindowListener`
- `MouseListener`
- `KeyListener`
- `ChangeListener`
- `XXXListener`

20

- *XXXListener* je vmesnik – zbirka metod, ki jih je potrebno implementirati za odziv na *XXXEvent*
- vsaka komponenta, ki se registrira kot poslušalec, mora implementirati vse metode ustreznega vmesnika
- adapterji so “prazna” implementacija poslušalcev
- pogosto za implementacijo poslušalca kreiramo podrazred adapterja in predefiniramo le posamezne metode
- anonimni notranji razredi so idealni za implementacijo poslušalcev in adapterje

21

## MouseListener

| Modifier and Type | Method                                   | Description                                                                           |
|-------------------|------------------------------------------|---------------------------------------------------------------------------------------|
| void              | <code>mouseClicked(MouseEvent e)</code>  | Invoked when the mouse button has been clicked (pressed and released) on a component. |
| void              | <code>mouseEntered(MouseEvent e)</code>  | Invoked when the mouse enters a component.                                            |
| void              | <code>mouseExited(MouseEvent e)</code>   | Invoked when the mouse exits a component.                                             |
| void              | <code>mousePressed(MouseEvent e)</code>  | Invoked when a mouse button has been pressed on a component.                          |
| void              | <code>mouseReleased(MouseEvent e)</code> | Invoked when a mouse button has been released on a component.                         |

```
public interface MouseListener
extends EventListener
```

The listener interface for receiving "interesting" mouse events (press, release, click, enter, and exit) on a component. (To track mouse moves and mouse drags, use the `MouseMotionListener`.)

The class that is interested in processing a mouse event either implements this interface (and all the methods it contains) or extends the abstract `MouseAdapter` class (overriding only the methods of interest).

The listener object created from that class is then registered with a component using the component's `addMouseListener` method. A mouse event is generated when the mouse is pressed, released clicked (pressed and released). A mouse event is also generated when the mouse cursor enters or leaves a component. When a mouse event occurs, the relevant method in the listener object is invoked, and the `MouseEvent` is passed to it.

22

## Dogodki (nižjenivojski in semantični)

`ComponentEvent` – proži se ob spremembi velikosti, pozicije, skritju ali prikazu komponente.

`FocusEvent` – Proži se, ko komponenta dobi ali zgubi fokus.

`InputEvent` – Nikoli se ne proži direktno. Služi kot organizacijski razred za vhodne dogodke

`KeyEvent` – Proži se ob zaznavi pritiska, sprostitve tipk ali specifične tipke

`MouseEvent` –ko se izvede v okviru komponente akcija z miško

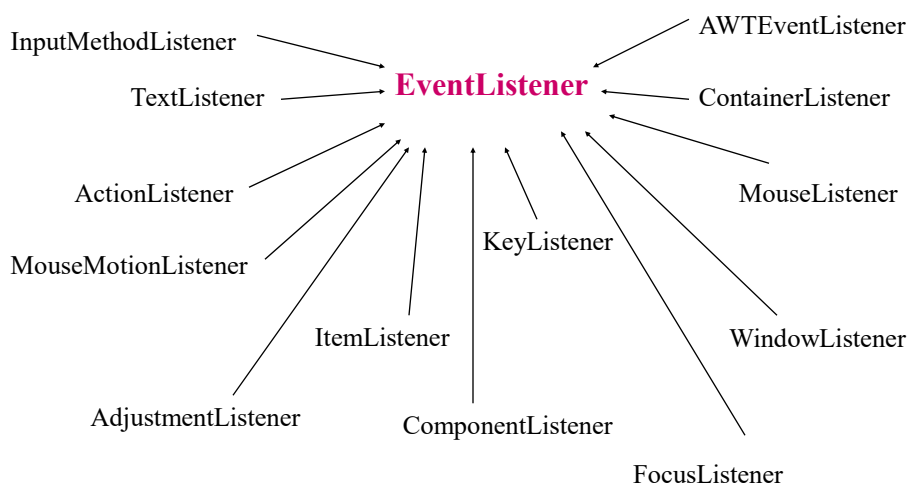
`ActionEvent` – Proži se ob osnovnih akcijah (pritisk gumba na miški ali tipkovnici).

`AdjustmentEvent` – Proži se ob nastavitvi vrednosti.

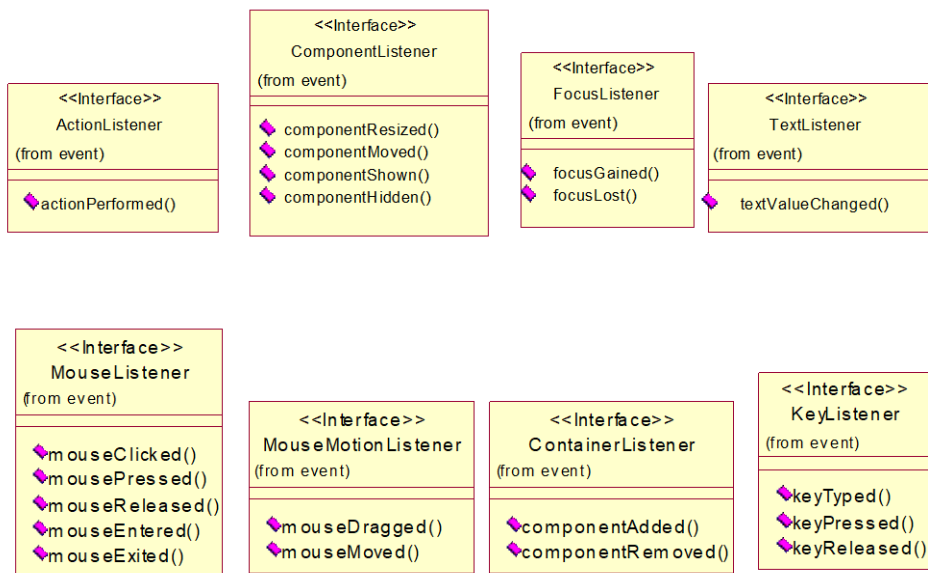
`ItemEvent` – Proži se ob spremembi stanja elementa.

23

## Poslušalci dogodkov implementirajo ustrezne vmesnike

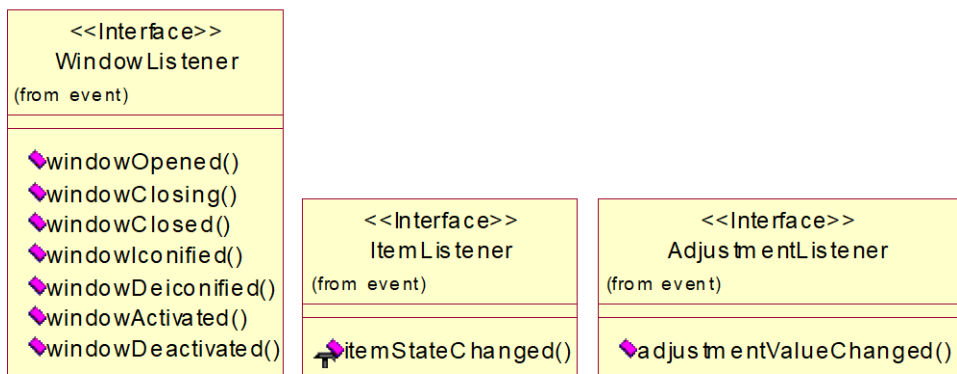


24



**Pazi:** na sliki niso navedeni parametri metod, npr. `ActionEvent`, `KeyEvent`, `MouseEvent`...

25



**Pazi:** na sliki niso navedeni parametri metod, npr. `WindowEvent`, ...

26

# Adapterji

## Adapterski razredi:

```
java.awt.event.ComponentAdapter  
java.awt.event.ContainerAdapter  
java.awt.event.FocusAdapter  
java.awt.event.KeyAdapter  
java.awt.event.MouseAdapter  
java.awt.event.MouseMotionAdapter  
java.awt.event.WindowAdapter
```

pogosto za implementacijo poslušalca kreiramo  
podrazred adapterja in predefiniramo le  
posamezne metode

27

# Adapter(ji)

- implementira(jo) ustrezni vmesnik
- vse metode imajo prazno telo
- so abstraktni (čeprav so vse metode implementirane)
- namesto, da sami implementiramo vse metode, dedujemo od ustreznega adapterja ter zagotovimo zgolj implementacijo posameznih metod
- ali obstajajo adapterji za vmesnike, ki imajo eno samo metodo?

28

```

class PoslusalecOkenskih implements WindowListener {
    public void windowActivated(WindowEvent e) {
    }

    public void windowClosed(WindowEvent e) {
    }

    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }

    public void windowDeactivated(WindowEvent e) {
    }

    public void windowDeiconified(WindowEvent e) {
    }

    public void windowIconified(WindowEvent e) {
    }

    public void windowOpened(WindowEvent e) {
    }
}

class PoslusalecOkenskih extends WindowAdapter {
    public void windowClosing(WindowEvent e) {
        System.exit(0);
    }
}

```



```

class PoslusalecMiske implements MouseListener {
    public void mouseClicked(MouseEvent e) {
    }

    public void mouseEntered(MouseEvent e) {
        System.out.println(" Miska vstopila");
    }

    public void mouseExited(MouseEvent e) {
        System.out.println(" Miska odšla");
    }

    public void mousePressed(MouseEvent e) {
    }

    public void mouseReleased(MouseEvent e) {
    }
}

class PoslusalecMiske extends MouseAdapter {
    public void mouseEntered(MouseEvent e) {
        System.out.println(" Miska vstopila");
    }

    public void mouseExited(MouseEvent e) {
        System.out.println(" Miska odšla");
    }
}

```



29

## Semantični dogodki

- So dogodki na višjem nivoju.
- Združujejo več nizkonivojskih dogodkov.
- Obravnavajo se identično kot nizkonivojski dogodki.
- Primer: ActionEvent, ItemEvent

30

## Action Event in awt gradniki

- Button  
"mouse click" na gumbu
- List  
dvojni klik na elementu seznama
- MenuItem  
klik na elementu izbire
- TextField  
pritisk tipke Return

31

## Item Event

- Checkbox  
označitev ali brisanje (uncheck)
- CheckBoxMenuItem  
element izbire označen ali odznačen
- Choice  
izbrana nova izbira
- List  
element seznama izbran ali deselektiran

32

## Variante poslušalcev

Objekt poslušalec:

- kot primerek **samostojnega razred**
- poslušalec kar vsebnik
- poslušalec primerek **notranjega (inner) razreda**
- poslušalec kot primerek **anonimnega notranjega razreda**

33

```
// panel vsebuje gumb, katerega
// labelo prikaže ob pritisku nanj
public class NoviPanel extends Panel {
    public NoviPanel () {
        ...
        Button b;
        add (b = new Button("Gumb"));
        b.addActionListener (new ButtonListener());
    }
}

class ButtonListener implements ActionListener {
    public void actionPerformed (ActionEvent e) {
        System.out.println(e.getActionCommand());
    }
}
```

objekt poslušalec primerek samostojnega razreda

34

```

public class Terminator implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent dogodek) {

        System.out.println(" sem obvescen" + dogodek.toString());
        // System.exit(0);

    }

}

public class FBIAgent implements ActionListener {

    String ime;

    public FBIAgent(String ime) {
        this.ime = ime;
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println(" FBI agent" + ime + " je obvescen + " + e.getSource()

    }

}

```

35

```

public class NoviPanel extends Panel {

    class NotranjiPoslusalec implements ActionListener {
        public void actionPerformed (ActionEvent e)
        {
            System.out.println("odziv notranjega
poslusalca");
        }
    }

    public NoviPanel (){
        ...
        Button b;
        add (b = new Button("Gumb"));
        b.addActionListener (new NotranjiPoslusalec());
    }

}

```

poslušalec primerek notranjega (inner) razreda

36

```

public class GUINovi {

    static class Tipkalni extends KeyAdapter {

        @Override
        public void keyPressed(KeyEvent e) {
            System.out.println(" nekaj si kliknil in sicer: "+e.getKeyCode() +
        }

        public void karneki() {}
    }
}

```

37

```

public class NoviPanel
    extends Panel implements ActionListener
{

    public void actionPerformed (ActionEvent e)
    {
        System.out.println("sam sem poslusalec");
    }

    public NoviPanel () {
        ...
        Button b;
        add (b = new Button("Gumb"));
        b.addActionListener (this);
    }
}

```

vsebnik sam je poslusalec

38

```

public class GUISamSvoj extends JFrame implements ActionListener, MouseMotionListener
{

    public GUISamSvoj() {

        JPanel vsebina = new JPanel();
        JButton gumb = new JButton(" Klikni me");
        vsebina.add(gumb);
        gumb.addActionListener(new NotranjiUhec());
        gumb.addActionListener(this);
        gumb.addMouseMotionListener(this);
    }
}

```

39

```

Button gumb = new Button("Ne stiskaj");

gumb.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Ne me stiskat");
    }
});

```

poslušalec kot edini primerek anonimnega notranjega razreda

40

## Anonimni notranji poslušalec

```

vnos.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println(" poslusam AE na TextFieldu");
        dodatnaVMojemANonimnemNotranjem();
    }

    public void dodatnaVMojemANonimnemNotranjem() {}

});

```

41

## Pozor!

Notranji (inner) objekti lahko dostopajo do atributov in metod zunanjega objekta, ta pa do vsega, kar imajo gnezdeni / vsebovani objekti.

Pri statičnih notranjih razredih veljajo identične omejitve kot smo že vajeni - static lahko namreč uporablja le statične metode in attribute zunanjega razreda.

42

## Več poslušalcev na isti komponenti (viru)

```

ActionListener poslusalec = new PoslusalecAkcij();
gumb1.addActionListener(poslusalec);
gumb1.addActionListener(new Terminator());
gumb1.addActionListener(new ActionListener() {

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Primerek notranjega anonimnega obveščen");
    }
});

class Terminator implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {

        class PoslusalecAkcij implements ActionListener {

            @Override
            public void actionPerformed(ActionEvent e) {

```

43

## En objekt posluša na več komponentah isto vrsto dogodka

```

class PoslusalecAkcij implements ActionListener {

    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println(" Izvor dogodka" + e.getSource());
    }
}

ActionListener poslusalec = new PoslusalecAkcij();

gumb1.addActionListener(poslusalec);
gumb2.addActionListener(poslusalec);
tf.addActionListener(poslusalec);

```

44

## En objekt posluša različne dogodke na različnih komponentah / virih

```
class MultiPoslusalec implements ActionListener, WindowListener, KeyListener, MouseListener {

class MultiPoslusalec extends WindowAdapter implements ActionListener, KeyListener, MouseListener {

    MultiPoslusalec multi = new MultiPoslusalec();

    gumb1.addActionListener(multi);
    tf.addKeyListener(multi);
    gumb2.addMouseListener(multi);
    okno.addWindowListener(multi);
```

45

## Dogodkovni model

- **objekt poslušalec** za določeno vrsto dogodka je **primerek razreda**, ki implementira ustrezni vmesnik
  - za ActionEvent -e vmesnik ActionListener
  - za KeyEvent -e vmesnik KeyListener
  - za WindowEvent -e vmesnik WindowListener
  - ...

Lahko je primerek / instanca:

- samostojnega (neodvisnega) razreda
- notranjega razreda
- anonimnega notranjega razreda
- ... this

46

# GRADNIKI SWING

Javax.swing

47

## Kaj so Swing gradniki?

- nabor prilagodljivih vizualnih gradnikov
- ob vpeljavi del JFC (Java Foundation Classes)
- Ime gradnikov: **J**Gradnik
- ni “peer” razredov na gostujočem sistemu (*lightweight komponente*)
- možnost zagotovitve enovitega izgleda (L&F) na različnih platformah
- uporaba vzorca MVC (Model View Controller)

48

## Nove možnosti

- Precej vgrajenih kontrol
  - gumbi-slike, zavihki, drsniki, orodjarne, izbirniki barve, HTML tekstovna področja, drevesa, tabele
- Možnosti prilagoditve
  - stili okvirjev (borders), poravnava besedila, osnovne možnosti risanja, sliko lahko dodamo skoraj vsaki kontroli
- Izgled
  - ni omejitve le na "native look & feel"
- Razne "malenkosti"
  - "double buffering", namigi (tool tips), uporabniški cursorji, kombinacije tipk (keyboard accelerators)
- arhitektura MVC (Model-View-Controller)
  - Interna predstavitev dreves, seznamov in tabel se lahko spremeni

49

## Swing vs. AWT

- Swing uporablja dosti sorodnih konceptov kot AWT
  - Enak dogodkovni model (delegacijski model)
  - Enake razporejevalnike gradnikov.
  - Mnogo Swing in AWT gradnikov ima skupne nadrazrede, npr. Component, Container in Window.

50

## awt vs swing

### awt komponente

- odvisne od platforme
- „heavyweight“
- ni podpore več L/F
- manj komponent

### swing komponente

- neodvisne od platforme
- „lightweight“
- različni L&F
- bogatejši nabor komponent

**MVC (Model-View-Controller)**

51

## Swing vs. AWT

- J – dogovor glede poimenovanja
- Lahke komponente (lightweight)
- `paintComponent()` za risanje (ne `paint()`)
- nov privzet izgled (Metal)
  - Eksplicitno je potrebno nastaviti domač (native) izgled
- **ne mešat v istem oknu Swing in awt komponent**

52

## JFrame

- imamo vsebinski panel (contentPane)
- Zapiranje
  - Okno se avtomatsko zapre
  - Zadnje ne pomeni zaključek izvajanja – to dosežemo z:  
`setDefaultCloseOperation(EXIT_ON_CLOSE);`
- Privzeti L&F
  - Java (Metal)

53

## swing

JFrame kot glavno okno

**Ne dodajamo neposredno na ta JFrame**, temveč naredimo objekt razreda JPanel (npr. `nasPanel`) in nato:

```
JFrame glavno = new JFrame();
JPanel nasPanel = new JPanel();
glavno.setContentPane(nasPanel);
// komponente dodajamo na nasPanel
```

ali pa pridobimo:

```
JPanel vsebina = (JPanel) glavno.getContentPane();
// komponente dodajamo na vsebina (ki je primerek razreda JPanel)
```

54

## Vsebnik JPanel

### **JPanel**

ContentPane

setBackgroundColor

setToolTipText

setBorder(BorderFactory.createLineBorder(Color....))

setOpaque (ni ali je transparentno)

55

## Standard Borders

setBorder(BorderFactory.\_\_\_\_\_)

createEmptyBorder(z,l,s,d);

createLineBorder(barva);

createLineBorder(barva, debelina)

createTitledBorder(niz)

...

56

```

import javax.swing.*;

public class GUISwing {

    public static void main(String[] args) {

        JFrame glavno = new JFrame(" Glavno okno");
        JPanel nasPanel = new JPanel();

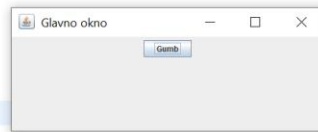
        glavno.setContentPane(nasPanel);
        nasPanel.add(new JButton(" Gumb "));

        glavno.setVisible(true);

        glavno.setSize(500,200);

    }
}

```



57

```

import javax.swing.*;

public class GUISwing {

    public static void main(String[] args) {

        JFrame glavno = new JFrame(" Glavno okno");
        JPanel nasPanel = (JPanel) glavno.getContentPane();

        nasPanel.add(new JButton(" Gumb "));

        glavno.setVisible(true);

        glavno.setSize(500,200);

    }
}

```



Pazi: upravljalac izgleda na nasPanel...

58

## Novosti

- JLabel
  - HTML vsebina, okvirji
- JButton
  - ikone, poravnava, mnemoniki
- JPanel
  - okvirji
- JSlider
  - označbe in labelce

...

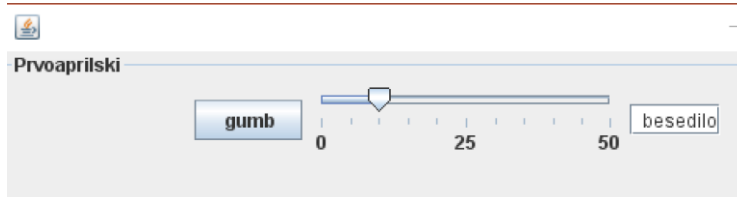
59

## JButton

```
ImageIcon slikca= new ImageIcon("cot.gif");  
JButton g = new JButton("ime",slikca);  
g.setHorizontalTextPosition  
    (SwingConstants.LEFT);
```

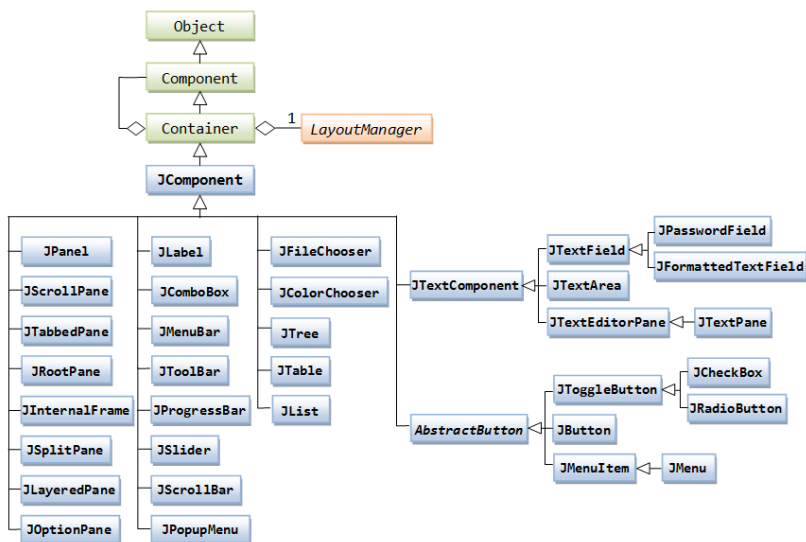
60

# JSlider



```
panel.add(drsnik = new JSlider(JSlider.HORIZONTAL,0,50,10));
drsnik.setPaintLabels(true);
drsnik.setPaintTicks(true);
drsnik.setMajorTickSpacing(25);
drsnik.setMinorTickSpacing(5);
```

61



62

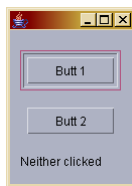
## Native L&F

```
public class WindowUtilities {

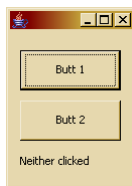
    public static void setNativeLookAndFeel ()
    {
        try {
            UIManager.setLookAndFeel (
                UIManager.getSystemLookAndFeelClassName());
        } catch (Exception e) {
            System.out.println("Error setting native LAF:");
        }
    }
}
```

63

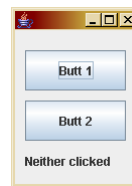
## Izgled (Look and feels)



CDE/Motif



Windows



Metal

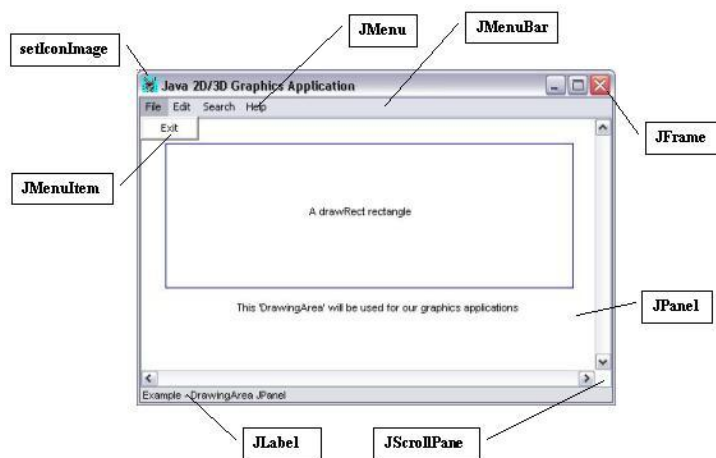
```
try {
    UIManager.setLookAndFeel (
        "com.sun.java.swing.plaf.motif.MotifLookAndFeel");
}
catch (Exception e)
{
    System.out.println("Težave pri pridobivanju L&F");
}
```

64

## Kateri so na voljo?

```
Object a[] = UIManager.getInstalledLookAndFeels();  
for (int i=0; i<a.length; i++)  
    System.out.println(a[i]);
```

65



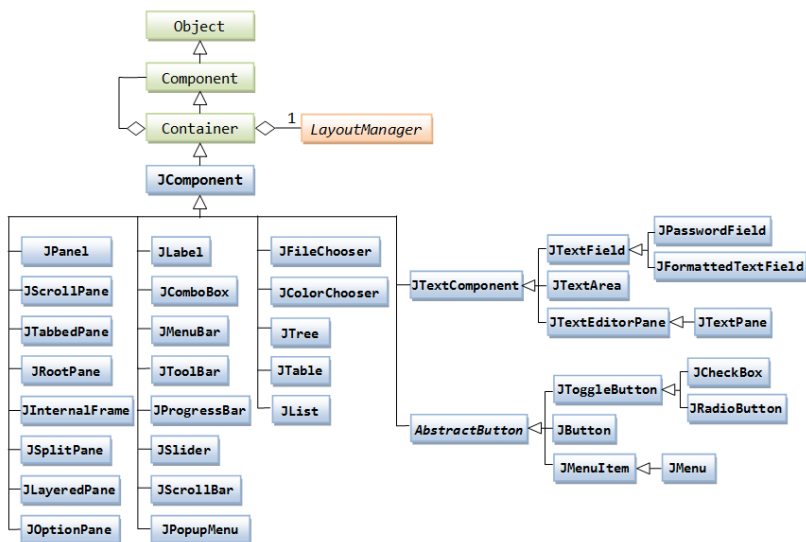
66

# Internal Frames

- MDI aplikacije
- Podpora v Swingu
  - **JDesktopPane**  
glavno okno, ki vsebuje vse ostale
  - **JInternalFrame**  
enako kot JFrame, le da mora biti znotraj JDesktopPane

```
JInternalFrame(title, resizable, closeable, maximizable,
iconifiable);
```

67



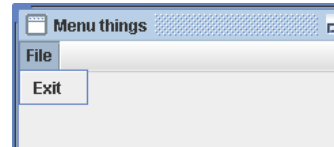
68

# Menu

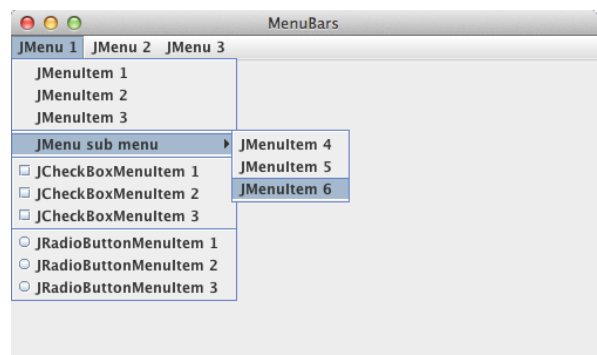
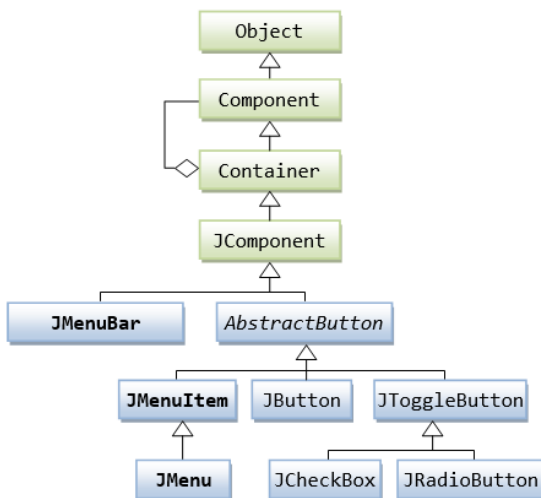
```

Main app = new Main();
..
JMenuBar myMenuBar = new JMenuBar();
JMenu menu1 = new JMenu("File");
JMenuItem item = new JMenuItem("Exit");
item.addActionListener(app);
menu1.add(item);
myMenuBar.add(menu1);
frame.setJMenuBar(myMenuBar);
..
public void actionPerformed(ActionEvent e)
{
    System.exit(0);
}

```



69



70

## Swing paketi

- Od Jave 1.2 dalje so paketi v javax.swing.\*.

| Paket                    | Opis                                                                                                                                 |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| javax.swing              | korenski paket za vse s Swing-om povezane pakete. Sam paket vsebuje najpomembnejše vmesnike in razrede, vključno z lahkimi gradniki. |
| javax.swing.event        | vsebuje vmesnike in razrede za specializirane dogodke, ki jih generirajo Swing gradniki                                              |
| javax.swing.plaf         | vsebuje podporo spremenljivemu izgledu (look & feel)                                                                                 |
| javax.swing.undo         | vsebuje podporo za uporabo undo in redo funkcionalnosti                                                                              |
| javax.swing.colorchooser | vsebuje gradnik JColorChooser                                                                                                        |
| javax.swing.filechooser  | vsebuje gradnik JFileChooser                                                                                                         |
| javax.swing.table        | vsebuje gradnik JTable                                                                                                               |
| javax.swing.tree         | vsebuje gradnik JTree                                                                                                                |
| javax.swing.text         | vsebuje podporo za tekstovne gradnike v Swing-u                                                                                      |

71

## Poslušalci in Swing

- 13 v java.awt.event
- 23 v javax.swing.event
- Skupno torej 36 vmesnikov!

72

# Notranji razredi

Inner / nested classes

73

```
import java.util.ArrayList;

public class Hisa {
    private String naziv;
    private ArrayList<Soba> sobe = new ArrayList<Hisa.Soba>();

    static class Nadstresnica {
        @Override
        public String toString() {
            return "Nadstresnica";
        }
    }

    class Soba {
        private String ime;

        Soba(String ime) {
            this.ime = ime;
            sobe.add(this);
        }

        @Override
        public String toString() {
            return "Sem " + ime + " v hisi" + naziv ;
        }
    }

    public Hisa(String naziv) {
        this.naziv = naziv;
    }

    @Override
    public String toString() {
        return sobe.toString();
    }
}

Hisa vila = new Hisa(" Vila Kamnica");
Hisa.Soba spalnica = vila.new Soba("spalnica");
Hisa.Soba otroška = vila.new Soba("otroška");

Hisa.Soba naVikendu = new Hisa(" Vikend na Pohorju").new Soba("Dnevna s kaminom");

new Hisa.Nadstresnica();
```

74

## Pozor!

Primerki notranjih (inner) razredov lahko **dostopajo do atributov in metod zunanjega objekta** (primerka outer razreda), ta pa do vsega v **objekti notranjega razreda**.

Pri **statičnih** notranjih, tj. gnezdenih (nested) razredih **veljajo** identične **omejitve** kot smo že vajeni - static lahko namreč uporablja le statične metode in attribute zunanjega razreda.

75

✗ Do `this$0` ne moreš neposredno dostopati v Javi:

- Ta spremenljivka ni dostopna iz Jave
- Je **synthetic field**, kar pomeni, da jo ustvari prevajalnik za interno uporabo.
- V **source kodi** Jave je ne moreš uporabiti (compiler ti ne bo dovolil).



```
Compiled from "Hisa.java"
class Notranji.Hisa$Soba {
    final Notranji.Hisa this$0;
    Notranji.Hisa$Soba(Notranji.Hisa, java.lang.String);
    public java.lang.String toString();
}
```

```
Compiled from "Hisa.java"
class Notranji.Hisa$Nadstresnica {
    Notranji.Hisa$Nadstresnica();
    public java.lang.String toString();
}
```

- Napredno:  
za pridobitev vrednosti  
uporabimo Reflection API 😊

```
public void printHisa() throws Exception {
    Field outerField = this.getClass().getDeclaredField("this$0");
    Hisa outer = (Hisa) outerField.get(this);
    System.out.println("Mama objekt: " + outer);
}
```

## The Reflection API

Reflection is commonly used by programs which require the ability to examine or modify the runtime behavior of applications running in the Java virtual machine. This is a relatively advanced feature and should be used only by developers who have a strong grasp of the fundamentals of the language. With that caveat in mind, reflection is a powerful technique and can enable applications to perform operations which would otherwise be impossible.

76